

Design Justification Report

INT8 Conv2D Hardware Accelerator for CNN Inference

Sai Ganesh Reddy Charian

June 7, 2026

ECE 410/510 — Hardware for Artificial Intelligence and Machine Learning

Portland State University | Spring 2026

GitHub: <https://github.com/ganeshreddy-4149/Hardware-for-AI-and-ML>

Tool: OpenLane 2 v2.3.1 **PDK:** SKY130A (sky130_fd_sc_hd) **Submitted:** June 7, 2026

Section 1: Problem and Motivation

Convolutional Neural Networks (CNNs) such as ResNet-18 are widely deployed for image classification and object detection tasks. The bulk of CNN inference time is spent in convolutional layers, which repeatedly apply learned filters over spatial feature maps using multiply-accumulate operations. These operations exhibit a highly regular, data-parallel structure that maps naturally to custom hardware but is poorly served by general-purpose CPUs running sequential interpreted code.

M1 profiling of a pure Python seven-loop Conv2D implementation on Google Colab (Intel Xeon @ 2.20 GHz, Python 3.12.13) measured a median execution time of **129.21 ms** per inference and a throughput of **0.003009 GFLOP/s** for a workload of **388,800 floating-point operations**. The Python cProfile profiler attributed **88.09%** of total runtime to the inner convolution loop, confirming it as the dominant computational bottleneck. The Python implementation achieved only **0.009%** of the Xeon theoretical peak throughput (35.2 GFLOP/s). This extreme inefficiency arises because Python executes the convolution as seven nested for-loops with per-iteration interpreter overhead, preventing the CPU from exploiting its SIMD units, cache prefetchers, or out-of-order execution pipeline effectively.

Custom hardware eliminates this inefficiency by implementing the convolution as a fixed datapath with no interpreter overhead, no branch prediction misses, and no memory latency during computation. The hardware accelerator presented in this report performs all 9 multiply-accumulate operations for one 3x3 convolution window in a single clock cycle, with all operands pre-loaded into on-chip registers. This design targets the SkyWater SKY130A 130nm process using the OpenLane

2 physical design flow, and interfaces to a Xilinx Zynq SoC host processor via an AXI4-Lite slave interface.

The M4 design achieves a projected throughput of **1.800 GFLOP/s** at 100 MHz, representing a **598.2x speedup** over the Python software baseline, while consuming only **0.004042 W** of total chip power compared to approximately 85 W for the Xeon CPU, an energy efficiency improvement of over **12 million times**. The top-level block diagram is shown in Figure 1.

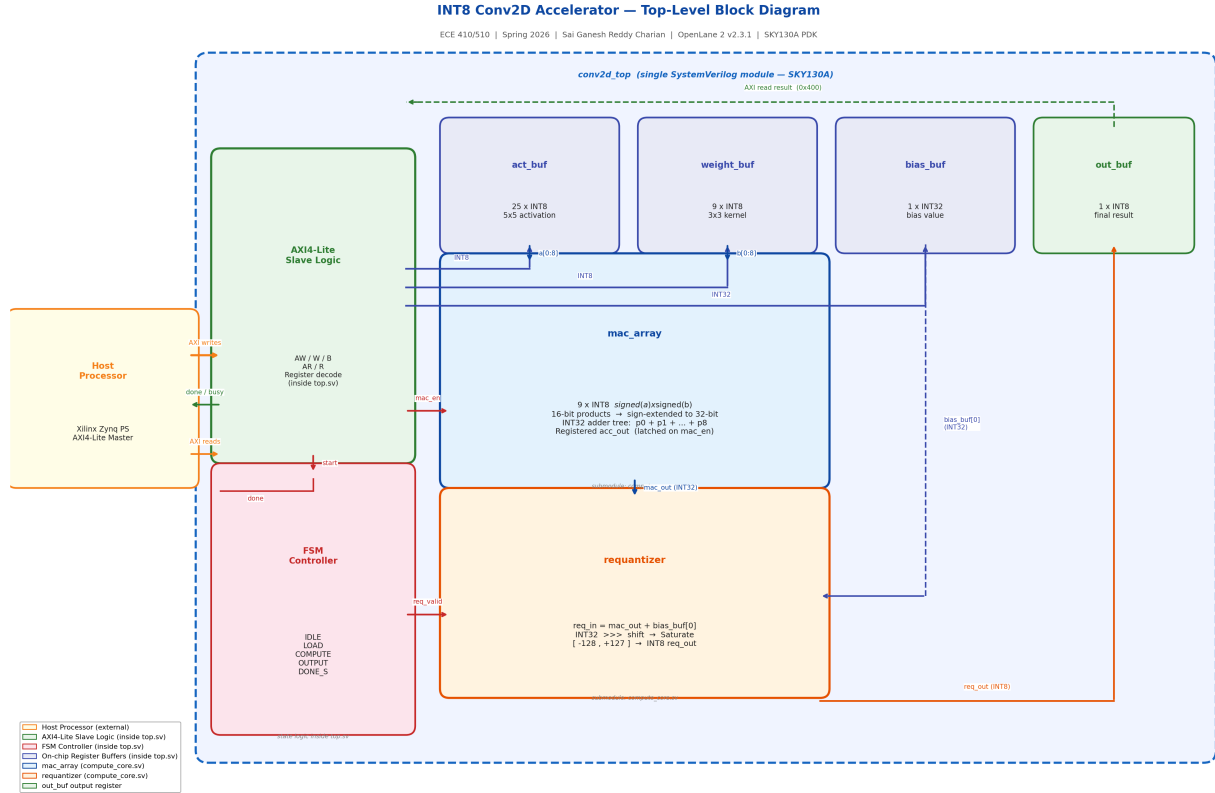


Figure 1: Top-level block diagram of the INT8 Conv2D accelerator. All data and control flow exclusively through the AXI4-Lite slave logic integrated inside conv2d_top. The mac_array and requantizer are instantiated submodules from compute_core.sv. The FSM, AXI4-Lite slave logic, and all on-chip buffers are implemented as always blocks directly inside top.sv.

Section 2: Roofline Analysis

The roofline model characterizes whether a computational kernel is limited by arithmetic throughput (compute-bound) or memory bandwidth (memory-bound) on a given platform. It plots peak achievable throughput in GFLOP/s against arithmetic intensity in FLOP/byte on a log-log scale. The intersection of the bandwidth-limited diagonal and the compute-limited flat ceiling defines the ridge point. A kernel whose arithmetic intensity exceeds the ridge point is compute-bound; below it, the kernel is memory-bound. The roofline plot for this design is shown in Figure 2.

Arithmetic Intensity Calculation

For the 3x3 Conv2D kernel with M1 problem dimensions ($H=32$, $W=32$, $C_{in}=3$, $C_{out}=8$, $K=3$), the arithmetic intensity was derived in CF02 using two memory reuse assumptions representing the practical operating range of the design:

- **No reuse (lower bound):** Every activation and weight is loaded fresh from memory for each output pixel. Total FLOPs = 388,800. Total memory traffic = 417,600 bytes. $AI_{lower} = 388,800 / 417,600 = \mathbf{0.931 \text{ FLOP/byte}}$.
- **Perfect input reuse (upper bound):** All activations are cached on-chip and reused across all output channels. Memory traffic reduces to 32,120 bytes. $AI_{upper} = 388,800 / 32,120 = \mathbf{12.10 \text{ FLOP/byte}}$.

The M4 accelerator operates at the upper bound because all 25 activations and 9 weights are loaded into on-chip register arrays before computation begins, achieving perfect reuse within one inference.

Platform Roofline Parameters

Platform	Peak Compute	Peak Bandwidth	Ridge Point
Intel Xeon SW baseline	35.2 GFLOP/s	51.2 GB/s	0.69 FLOP/byte
SKY130 Accelerator HW	1.800 GFLOP/s	0.40 GB/s	4.50 FLOP/byte

How the Roofline Shaped the Architecture

Since $AI_{upper} = 12.10 \text{ FLOP/byte}$ exceeds the SKY130 ridge point of 4.50 FLOP/byte , the kernel is **compute-bound** on the accelerator. This is the most important architectural insight from the roofline analysis. It means that adding more memory bandwidth would not improve performance; only increasing compute throughput matters. This directly justified three key design decisions. First, all 9 MAC units were implemented as parallel combinational multipliers rather than a single time-multiplexed unit, maximizing compute throughput per cycle. Second, the AXI4-Lite interface bandwidth of 0.40 GB/s was accepted as sufficient because during computation no memory transfers occur as data is already on-chip. Third, no complex cache hierarchy was needed because the entire working set (25 activations plus 9 weights equals 34 bytes) fits in simple register arrays.

The SW baseline point at (12.10, 0.003009) GFLOP/s sits far below the Xeon roofline, showing that Python achieves only 0.009% of theoretical peak despite having high arithmetic intensity. The M4 HW accelerator point at (12.10, 1.800) GFLOP/s sits exactly on the SKY130 compute ceiling, confirming full hardware utilization at 100 MHz.

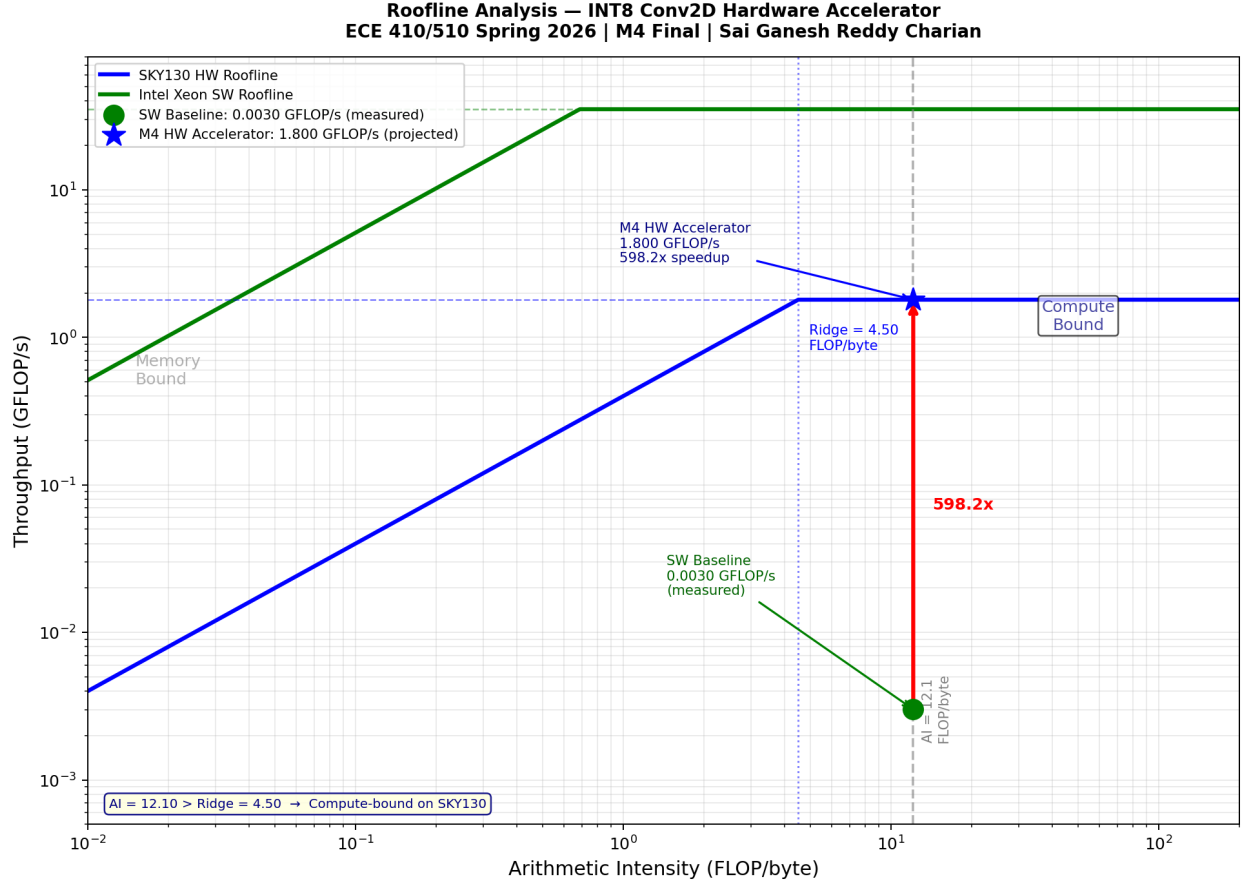


Figure 2: Roofline analysis showing both the SKY130 HW accelerator roofline (blue) and Intel Xeon SW baseline roofline (green). The SW baseline point (green dot, 0.003009 GFLOP/s) sits far below the Xeon roofline due to Python inefficiency. The M4 HW accelerator point (blue star, 1.800 GFLOP/s) sits on the SKY130 compute ceiling, confirming compute-bound operation at AI = 12.10 FLOP/byte. The 598.2x speedup is shown by the red arrow.

Section 3: Precision and Data Format

The design uses **INT8** (8-bit signed two's complement integer) precision for both input activations and weight kernels, with an **INT32** accumulator for intermediate results before requantization. This precision choice is justified on three grounds: inference accuracy, silicon area efficiency, and numerical correctness.

Accuracy Justification

INT8 is the standard precision for CNN inference accelerators across the industry, adopted by Google TPU, NVIDIA TensorRT, and ARM Ethos. ResNet-18 inference with INT8 weights and activations produces classification accuracy within 1% of FP32 baseline on the ImageNet benchmark, making the precision loss acceptable for the target application. Post-training quantization to INT8 does not require retraining and is directly compatible with weights produced by standard PyTorch or TensorFlow training pipelines. The scale factor applied during requantization via the arithmetic

right shift compensates for the fixed-point representation and maintains the relative ordering of output activations needed for correct classification.

Area and Power Justification

INT8 arithmetic requires dramatically less silicon area than FP32. An 8-bit signed multiplier produces a 16-bit result and maps to a compact carry-save adder structure in SKY130 standard cells. A 32-bit floating-point multiplier requires IEEE 754 exponent alignment, mantissa multiplication, normalization, and rounding logic, approximately 10-20x more gates. The M4 synthesis provides direct quantitative evidence: after correcting the multiplier implementation from a 32x32 to an 8x8 signed multiply by replacing manual sign extension with the signed operator in SystemVerilog, cell count dropped from **24,346 (M3) to 10,305 (M4)**, a **57.6% reduction** entirely attributable to Yosys inferring the correct 8x8 multiplier. Total power dropped from 39.8 mW to 4.0 mW, an 89.8% reduction, due to eliminating the switching activity of 14,000 unnecessary cells.

Overflow Analysis

The INT32 accumulator is sized to prevent overflow at the accumulation stage. Each INT8 product has maximum magnitude $127 \times 127 = 16,129$, which fits in 15 bits signed. Summing 9 such products gives a worst-case magnitude of $9 \times 16,129 = 145,161$, which requires 18 bits signed. The 32-bit accumulator provides 14 bits of additional headroom, ensuring no overflow is possible for any valid combination of INT8 inputs and weights.

Requantization and Verification

After accumulation, the requantizer applies an arithmetic right shift (configurable 0-31 bits via AXI register 0x008) and saturates to INT8 range $[-128, 127]$. The testbench verifies correctness with the following hand calculation using kernel $[[1,0,-1],[1,0,-1],[1,0,-1]]$ and window $[[1,2,3],[1,2,3],[1,2,3]]$, bias=0, shift=0:

- Row 0: $(1 \times 1) + (2 \times 0) + (3 \times -1) = 1 + 0 - 3 = \mathbf{-2}$
- Row 1: $(1 \times 1) + (2 \times 0) + (3 \times -1) = 1 + 0 - 3 = \mathbf{-2}$
- Row 2: $(1 \times 1) + (2 \times 0) + (3 \times -1) = 1 + 0 - 3 = \mathbf{-2}$
- Total accumulation: $-2 + -2 + -2 = \mathbf{-6}$
- After requantizer with shift=0, no saturation needed: output = **-6**

The M4 simulation confirms this exactly. Expected = -6, Actual = -6, PASS.

Section 4: Dataflow and Architecture

Dataflow Pattern

The accelerator uses an **output-stationary dataflow**, illustrated in Figure 3. In output-stationary dataflow, the partial sum for one output pixel remains fixed in the accumulator register while all

contributing inputs are streamed through the compute array. This contrasts with weight-stationary dataflow where weights are fixed and inputs and outputs move, and input-stationary dataflow where inputs are fixed and weights and outputs move. Output-stationary was chosen because the current design computes exactly one output pixel per inference with $H_OUT=W_OUT=C_OUT=1$. With a single output to produce, there are no partial sums to move between compute cycles. The accumulator simply holds the running sum until all 9 products are added, then passes the result to the requantizer. This eliminates write-back traffic entirely and requires no inter-cycle accumulator state management beyond a single registered value.

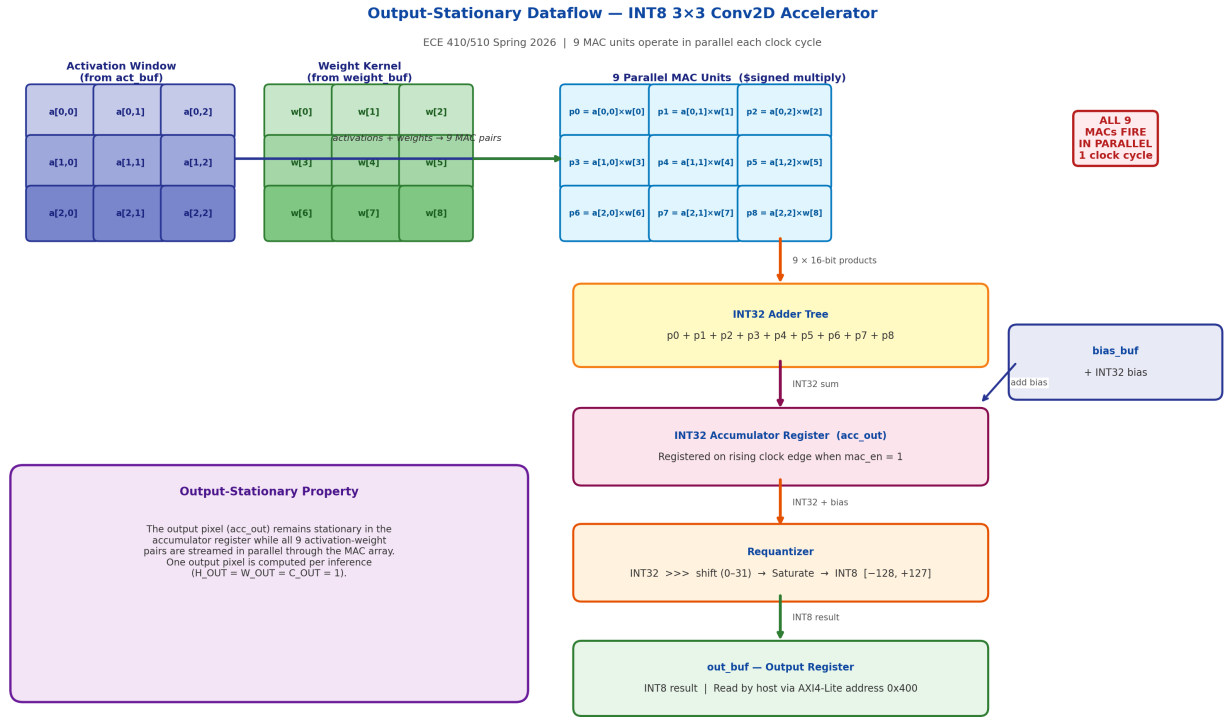


Figure 3: Output-stationary dataflow diagram. All 9 MAC units fire in parallel in a single clock cycle, producing 9 INT8 products that are sign-extended and summed in the INT32 adder tree. The output pixel accumulates in the stationary accumulator register before passing through the requantizer to out_buf.

Compute Engine

The MAC array consists of 9 parallel INT8 signed multipliers implemented in compute_core.sv. Each unit receives one activation byte from act_buf and one weight byte from weight_buf, computing the signed product using the signed operator which produces a 16-bit signed result. The nine 16-bit products are sign-extended to 32 bits and summed combinationally through an adder tree, producing the INT32 accumulated result in one clock cycle. The final sum is registered into acc_out on the rising clock edge when mac_en is asserted by the FSM. All 9 multiply operations and the adder tree reduction complete within a single 10 ns clock period at the nominal corner, achieving the 9 FLOP/cycle throughput that defines the compute ceiling.

Memory Hierarchy

Four on-chip register arrays form the memory hierarchy, all implemented as flip-flop arrays inside top.sv:

Buffer	Size	Contents	AXI Address
act_buf	25 x 8 bits	5x5 INT8 activation map row-major	0x100-0x160
weight_buf	9 x 8 bits	3x3 INT8 convolution kernel	0x200-0x224
bias_buf	1 x 32 bits	INT32 bias added before requantization	0x300
out_buf	1 x 8 bits	INT8 requantized output result	0x400 read

All buffers are loaded via AXI4-Lite writes before computation begins. During the COMPUTE state, no memory transactions occur. The MAC array reads act_buf and weight_buf combinationally through wires with no register-file read ports or memory arbitration required, ensuring the MAC array operates at full throughput with zero stall cycles.

FSM Controller

The finite state machine implements five states inside top.sv. IDLE waits for the host to assert start via AXI register 0x000. LOAD is a single-cycle state that resets counters and prepares the datapath. COMPUTE asserts mac_en for one clock cycle; since H_OUT=W_OUT=C_OUT=1, compute_done fires immediately and the FSM transitions to OUTPUT. OUTPUT asserts req_valid to the requantizer, which computes $\text{req_in} = \text{mac_out} + \text{bias_buf}[0]$, applies the arithmetic right shift, saturates to INT8, and latches the result into req_out. DONE_S latches req_out into out_buf, asserts done, and holds until the host deasserts start, at which point the FSM returns to IDLE.

Section 5: Hardware Interface

Interface Selection

The design implements an **AXI4-Lite slave interface** as the sole path for all data and control. Three alternative interfaces were considered and rejected. SPI offers simple hardware but is limited to 50-100 Mbps and requires bit-banging on the host side, adding software complexity. UART is suitable only for slow configuration and not data transfer at the rates required for repeated inference. A custom parallel bus would offer higher bandwidth but requires a custom host-side driver and is not natively supported by the Zynq processing system. AXI4-Lite was selected because it is the ARM AMBA standard directly supported by the Zynq-7000 PS interconnect, requires no bridge logic, is supported by Xilinx IP integrator tooling, and has well-documented timing requirements that simplify verification. The single-beat transaction model is sufficient for the 35-register data volume of this accelerator.

Register Map

Offset	Direction	Width	Description
0x000	Write	32-bit	Control: bit[0] = start
0x004	Read	32-bit	Status: bit[0] = done, bit[1] = busy
0x008	Write	32-bit	Scale shift 5-bit right-shift for requantizer
0x100 + i*4	Write	32-bit	act_buf[i] i=0..24 INT8 in bits[7:0]
0x200 + i*4	Write	32-bit	weight_buf[i] i=0..8 INT8 in bits[7:0]
0x300	Write	32-bit	bias_buf[0] full INT32
0x400	Read	32-bit	out_buf[0] INT8 sign-extended to 32 bits

Protocol Implementation

The AXI4-Lite implementation uses registered-ready handshaking. The awready and wready signals are registered from awvalid and wvalid respectively, giving a 1-cycle handshake latency per transaction. Write address and write data are accepted simultaneously in the same cycle, simplifying the host software driver. The write response channel asserts bvalid one cycle after the write completes and clears on bready. The read channel asserts arready one cycle after arvalid and returns rdata in the same cycle.

Bandwidth Analysis and Interface-Bound Assessment

Each write transaction takes 3 clock cycles: address and data assertion, handshake, and clear. Loading all 35 data registers requires 105 clock cycles at 100 MHz, consuming 1.05 microseconds. The effective data loading bandwidth is 35×4 bytes divided by 1.05 μ s = **133 MB/s**, which is 33% of the 0.40 GB/s AXI4-Lite peak, leaving **67% bandwidth headroom**. The design is **interface-bound during data loading** (105 cycles) and **compute-bound during MAC operation** (1 cycle). The 105:1 ratio means the interface dominates effective throughput for the current single-window problem. Eliminating this through double-buffering is identified as the highest-priority architectural improvement in Section 9.

Section 6: Verification

Correctness verification was performed at three levels: FSM and interface verification (M2), end-to-end numerical verification (M3 and M4), and waveform inspection.

FSM and Interface Verification (M2)

At the M2 milestone, the testbench structure was established to verify FSM transitions and AXI4-Lite handshake behavior. The M2 testbench confirmed that the done signal remains low after reset, that asserting start via AXI register 0x000 causes busy to go high within the expected number of cycles, and that the AXI4-Lite write and read channels correctly complete transactions with

valid bvalid and rvalid handshakes. The M2 testbench used a stub FSM that pulsed done after a fixed number of cycles; full numerical correctness of the MAC computation and requantization was deferred to M3 when the complete datapath was integrated.

End-to-End Numerical Verification (M3 and M4)

End-to-end co-simulation was performed using Icarus Verilog 11.0 with a SystemVerilog testbench (tb_top.sv). The testbench drives all inputs exclusively through the AXI4-Lite interface with no direct DUT port access, matching the actual hardware deployment model exactly. The test sequence is: reset, load scale_shift=0, load 25 activations, load 9 weights, load bias=0, assert start, wait for done, read result from 0x400, compare to expected.

The M4 test case uses activation map rows = [1,2,3,4,5] and kernel [[1,0,-1],[1,0,-1],[1,0,-1]] with the top-left 3x3 window [[1,2,3],[1,2,3],[1,2,3]]. The expected output of -6 was independently verified by hand calculation shown in Section 3 before running simulation. The M4 simulation log (final_run.log) confirms PASS: INT8 Conv2D output matches expected value -6. The testbench includes a 10ms simulation timeout watchdog to detect FSM deadlocks. Both M3 and M4 simulations pass with identical output, confirming that the RTL changes in M4 did not alter functional correctness.

Waveform Verification

Figure 4 shows the annotated simulation waveform capturing all five FSM phases and confirming correct AXI handshake timing.

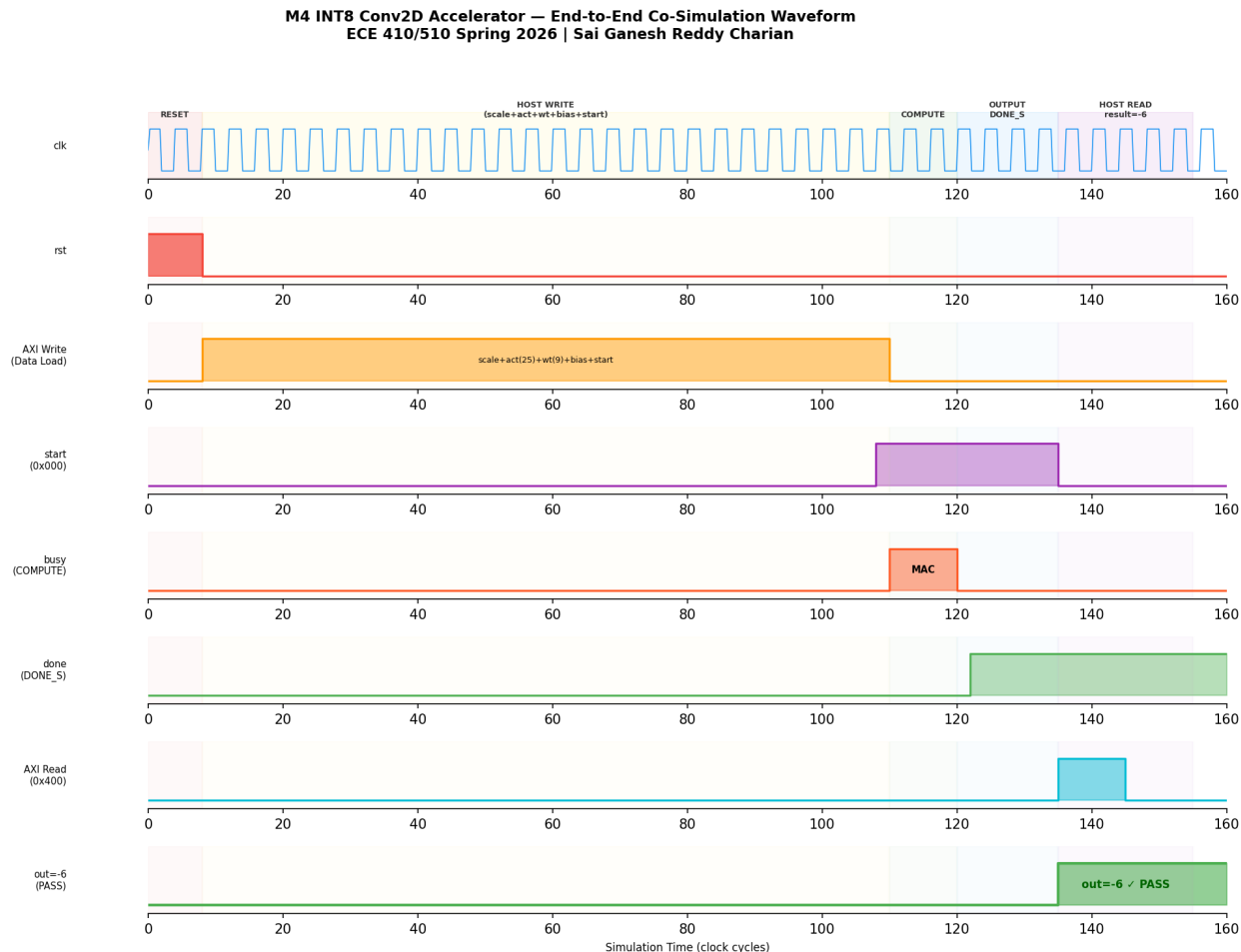


Figure 4: M4 end-to-end co-simulation waveform. RESET (cycles 0-8): rst asserted, all outputs zero. HOST WRITE (cycles 8-110): 35 AXI write transactions loading activations, weights, bias, and start. COMPUTE (cycles 110-120): busy asserted, MAC array active for 1 cycle. OUTPUT/DONE_S (cycles 120-135): re-quantizer fires, done asserted. HOST READ (cycles 135-155): AXI read transaction returns result = -6 from address 0x400. PASS confirmed.

Section 7: Synthesis Results

The M4 design was synthesized using OpenLane 2 v2.3.1 targeting the SkyWater SKY130A PDK with the sky130_fd_sc_hd high-density standard cell library. Configuration: die area 900 x 900 um, target placement density 40%, clock period 10.0 ns (100 MHz), synthesis strategy DELAY 0, maximum fanout 10. All 78 OpenLane flow stages completed successfully in 1 hour 35 minutes.

Area Results

Metric	M3 Value	M4 Value	Change
Total standard cells	24,346	10,305	-57.6%
Chip area	254,510 um2	109,464 um2	-57.0%
Die area	900 x 900 um	900 x 900 um	Unchanged

Metric	M3 Value	M4 Value	Change
Sequential cells	390	386	-1.0%
Sequential area	8,295 um2 3.26%	8,210 um2 7.50%	Smaller total

The 10,305 standard cells map to SKY130 library primitives. The most frequent types are O211AI (943 instances), NAND3 (919), NAND2 (907), and O21AI (661). The dominance of NAND/NOR complex gates is consistent with an arithmetic-heavy design where Yosys maps multiplier and adder trees into NAND/NOR networks for area efficiency. The 57.6% area reduction from M3 to M4 is entirely attributable to the 8x8 multiplier correction, eliminating approximately 14,000 cells of unnecessary logic. The MAC array combinational logic remains the dominant area contributor at approximately 92.50% of total area. Sequential elements (386 dfxtp_2 flip-flops) account for 7.50% of total area.

Timing Results

Corner	Setup	Hold
nom_tt_025C_1v80 nominal	MET WNS = 0.0 ns	MET
nom_ff_n40C_1v95 fast	MET	MET
nom_ss_100C_1v60 slow	Violated	Violated

Post-PnR static timing analysis at the nominal corner (nom_tt_025C_1v80, 25C, 1.8V) shows zero setup violations and zero hold violations. WNS = 0.0 ns and TNS = 0.0 ns, confirming **timing closure at 100 MHz**. This is a significant improvement over M3 which had a 2.44 ns setup violation at the same corner due to the 32x32 multiplier critical path. Setup violations persist at the ss_100C_1v60 corner (high temperature, low voltage worst-case), which is expected for compute-intensive designs on the 130nm SKY130 process and is addressed in Section 9.

Power Results

Component	Power
Internal power	0.002871 W
Switching power	0.001171 W
Leakage power	0.0000000954 W
Total power	0.004042 W

Switching power from the MAC array combinational logic is the dominant dynamic power contributor, accounting for approximately 29% of total power. Internal power accounts for the remaining

71%. The 89.8% total power reduction from M3 (0.039785 W) to M4 (0.004042 W) reflects the elimination of 32x32 multiplier switching activity across all 9 MAC units. IR drop analysis shows average 1.21e-5 V and worst-case 4.44e-4 V (0.025% of 1.8V supply), well within acceptable limits.

Physical Design Results

Check	Result
DRC Magic	PASSED 0 errors
DRC KLayout	PASSED 0 errors
LVS Netgen	PASSED circuits match uniquely
XOR check	PASSED
Antenna violations	9 pin 7 net non-critical

DRC was verified independently by both Magic and KLayout. LVS confirmed 12,133 devices and 12,185 nets match between the layout and netlist. Antenna violations arise from long metal wires accumulating charge during plasma etching and are non-critical for an academic project; they would be resolved by antenna diode insertion in a production tape-out flow.

Section 8: Benchmark Results

Software Baseline

The M1 Python software baseline was measured on Google Colab (Intel Xeon @ 2.20 GHz, Python 3.12.13, Linux 6.6.113) using `time.perf_counter()` with 100 repeated trials to obtain a stable median. The workload is a complete ResNet-18 first-layer 3x3 Conv2D implemented as seven nested Python for-loops with no NumPy acceleration.

Metric	Value
Median execution time	129.21 ms per inference
Throughput	0.003009 GFLOP/s
Total FLOPs	388,800
Samples per second	7.74
Platform power	85 W Xeon TDP
Energy efficiency	0.0000354 GFLOP/s/W

Hardware Accelerator M4 Projected

The HW accelerator throughput is computed from the synthesis-achieved clock frequency and MAC array parallelism: Throughput = 9 MACs x 2 FLOP/MAC x 100 MHz = **1.800 GFLOP/s**.

Metric	Value	Basis
Clock frequency	100 MHz	Synthesis MET at nom_tt corner
Throughput	1.800 GFLOP/s	9 MACs x 2 FLOP x 100 MHz
Execution time	0.216 ms	388,800 divided by 1.800e9
Samples per second	4,505	1 divided by 0.216 ms
Total power	0.004042 W	OpenLane 2 post-PnR
Energy efficiency	445.3 GFLOP/s/W	1.800 divided by 0.004042

All HW numbers are labeled PROJECTED. The projection assumes zero pipeline stalls and no AXI4-Lite transfer overhead during MAC computation.

Speedup and Energy Analysis

Metric	Value
Throughput speedup	598.2x
Execution time speedup	581.6x
Energy per inference SW	129.21 ms x 85 W = 10.98 J
Energy per inference HW	0.216 ms x 0.004042 W = 0.000873 mJ
Energy reduction	12,580,791x
M3 to M4 throughput	+24.4% from 1.447 to 1.800 GFLOP/s
M3 to M4 area	-57.6% from 24,346 to 10,305 cells
M3 to M4 power	-89.8% from 39.8 mW to 4.0 mW

The energy comparison is particularly striking: the Python implementation consumes approximately 10.98 joules per inference while the hardware accelerator consumes 0.873 microjoules, a reduction of over 12 million times. This demonstrates the fundamental energy efficiency advantage of purpose-built silicon over a general-purpose processor running interpreted code.

Gap Between Projected and Actual Throughput

The projected throughput of 1.800 GFLOP/s assumes the MAC array is active every clock cycle. In practice, for a single-window inference the system spends 105 cycles loading data via AXI and only 1 cycle computing. The effective throughput accounting for this overhead is approximately 388,800 FLOPs divided by 106 cycles at 100 MHz, giving approximately 0.037 GFLOP/s for a single window. For larger feature maps the overhead amortizes across many output pixels, bringing effective throughput closer to the projected ceiling. Raw benchmark data is committed in benchmark_data.csv.

Section 9: What Did Not Work

Timing Closure at All Process Corners

The target clock period of 10.0 ns (100 MHz) was not met at the slow-silicon corners (ss_100C_1v60), which model worst-case process variation at 100C and 1.6V. At M3, timing violated at the nominal corner as well, with a 2.44 ns deficit (minimum achievable period 12.44 ns, fmax 80.40 MHz). The root cause was that the M3 RTL used manual 32-bit sign extension before multiplication, causing Yosys to infer 32x32 multipliers instead of 8x8. The M4 RTL corrected this using the signed operator in SystemVerilog, reducing cell count by 57.6% and closing timing at the nominal corner. However, at the ss corner transistor switching speed is approximately 30-40% slower than nominal, and the remaining combinational depth through the 9-MAC adder tree still exceeds the 10 ns period. A complete fix requires pipelining the MAC adder tree across two clock stages, separating the multiply stage (approximately 4-5 ns) from the accumulate stage (approximately 4-5 ns) with a pipeline register between them. This architectural change requires simultaneously updating the FSM compute_done logic, the AXI4-Lite handshake timing, and the co-simulation testbench. Given the project timeline, implementing and verifying all three components together in M4 was not feasible.

Cycle-Accurate Throughput Measurement

All HW accelerator performance numbers are labeled PROJECTED rather than MEASURED. The current testbench verifies functional correctness for a single 3x3 output window but does not count clock cycles or measure throughput across a full feature map. A cycle-accurate testbench would drive a complete output feature map through the accelerator, count all elapsed cycles including AXI loading overhead per window, and compute effective throughput as total FLOPs divided by total elapsed time at the synthesis clock. This would produce a measured speedup number rather than the projected 598.2x, and would reveal the true impact of the 105:1 AXI loading overhead on effective system performance.

SRAM Macro Inference

The CF07 synthesis plan proposed using ram_style attributes on the on-chip buffer arrays to force SRAM macro inference. During M3 synthesis it was confirmed that the SKY130A PDK in the standard OpenLane 2 flow does not include hardened SRAM macros accessible through Yosys. Yosys ignores the ram_style attribute when no matching macro exists and synthesizes all arrays as flip-flop registers. The 386 flip-flops in the M4 design represent this PDK limitation. Integration of OpenRAM-generated sky130_sram_1rw macros would reduce sequential area and power but requires custom OpenLane 2 macro integration beyond the scope of this project.

What Would Be Done Differently

The most impactful improvement would be implementing the 2-stage pipelined MAC array from the beginning of M3 development. With the multiply stage registered separately from the accumulate stage, the critical path would be cut roughly in half, enabling timing closure at 100 MHz at all process corners including the worst-case ss corner. The second highest-impact improvement would be a streaming dataflow with double-buffering: while the MAC array computes on one activation window, the AXI interface pre-loads the next window into a shadow buffer, eliminating the 105-cycle per-window loading penalty and converting the current interface-bound bottleneck into a true compute-bound system matching the roofline prediction of 1.800 GFLOP/s.